

// 45 MIN – LIVE DEMO INCLUÍDA

VIBE HACKING.

▸ encontrando 0days ao vivo.

\$ WHOAMI



whoami

• • • ~/TALK — BASH

\$ whoami

Caio Lüders/@caioluders

Security Researcher

<https://lude.rs>

\$ cat bio.txt

↳ 15 anos quebrando coisas

↳ ElectroVolt / Cure53

↳ escrevo poesia e faço umas arte

\$ █

EXP

15

anos quebrando
coisas

HOJE

C53

ElectroVolt /
Cure53

FOCO

0day

research ofensivo

FORA

arte

poesia + arte

\$ CAT ROTEIRO.MD

o roteiro.

01 vocabulário

MCP · harness · skills · agent loop

02 filosofias

broad-first × depth-first · a matriz

03 anatomia de uma análise

mapear → triagem → source→sink → prova

04 exemplos do mundo real

0day · bug bounty · CTF · pentest

05 demo ao vivo

alvo escolhido pela plateia · sem rede

01

PARTE 01 / 05

vocabulário
obrigatório.

MCP

MODEL CONTEXT PROTOCOL

a **API** do seu modelo.

// O QUE FAZ

padrão aberto pra expor **ferramentas** ao modelo:
burp, semgrep, grep, ghidra, banco de dados,
navegador.

// ANTES VS DEPOIS

antes: glue code custom pra cada integração.
depois: plug-and-play. servidor MCP > modelo >
ação.

// PRA QUÊ

sair do "chat só de texto" e dar *mãos* pro
modelo. é onde o ganho real mora.

HARNESS

O EXOESQUELETO

o modelo é o **motor**. o harness é o **chassi**.

// O QUE É

o código que *envolve* o modelo: loop principal, gestão de contexto, retry, paralelismo, logs, budget de tokens.

// A VERDADE IMPOPULAR

you ganha 10x mais escrevendo harness decente do que trocando de modelo.

// RED FLAG

quem só fala de "qual modelo usar" geralmente não tem harness.

// NA PRÁTICA

Claude Code

Codex CLI

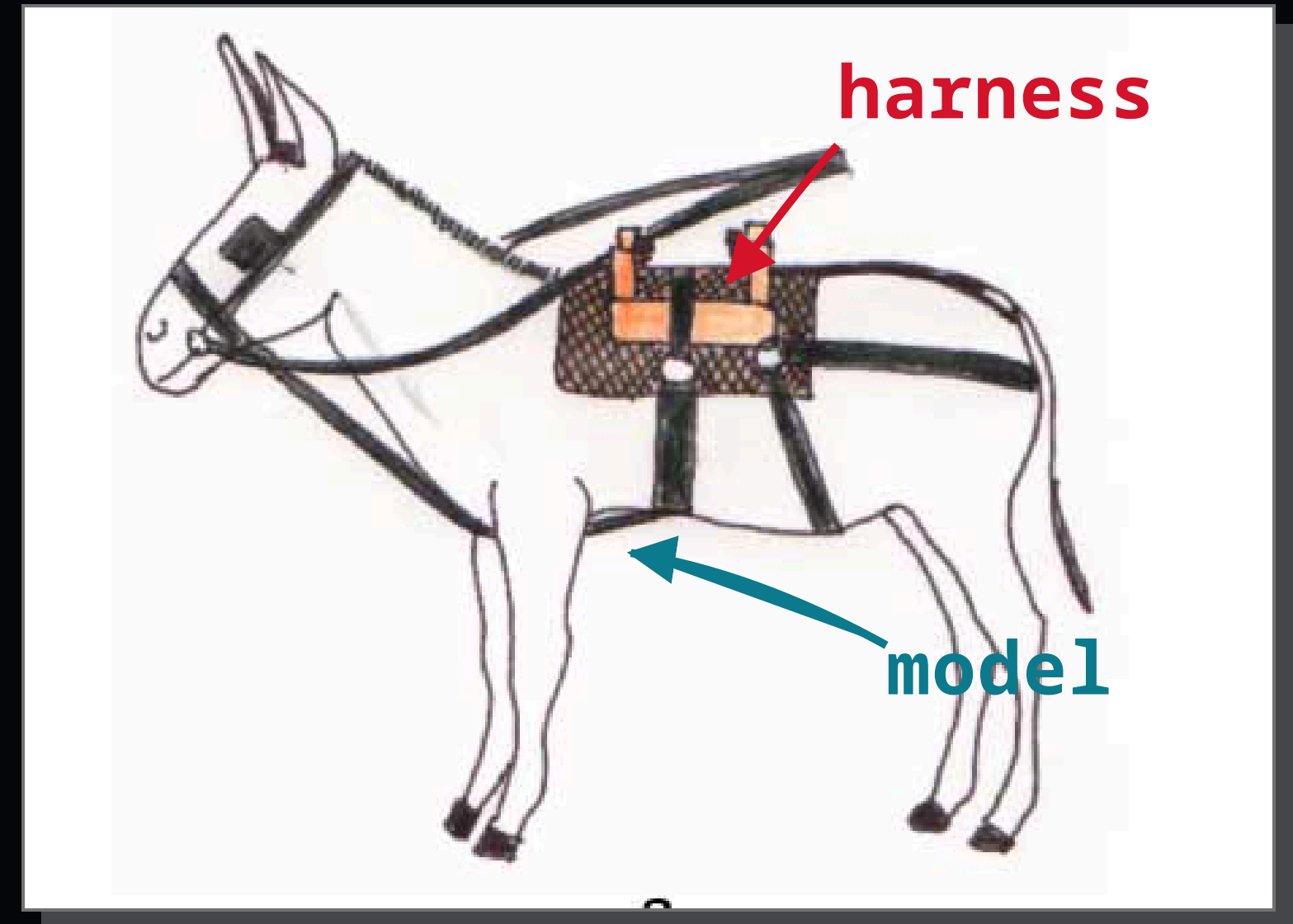
Pi

Droid

Hermes

OpenClaw

SWE-agent



// o motor sozinho não carrega nada

SKILLS

RUNBOOKS PRO MODELO

markdown. **pequeno**. focado.

// FORMATO

arquivo .md curto. "quando eu te der X, faça Y, Z." sem firula. sem RAG mirabolante.

// EXEMPLOS

↳ reverse-elf-funcs.md – extrair, ranquear, descrever

↳ jwt-quick-audit.md – auditar token em 4 passos

↳ ssrf-hypothesis.md – checklist pra cheiro de SSRF

// EFEITO COMPOSTO

quem tem *biblioteca* boa de skills hoje, hackeia em escala. tudo o resto é cosplay.



AGENT LOOP

THINK → ACT → OBSERVE → REPEAT

parece bobo. **é** bobo.
funciona.

// NO MUNDO REAL

ReAct

CodeAct

Reflexion

Plan-Execute

Ralph Wiggum

autoresearch

/goal

• • • LOOP.PY

```
# o ralph. burro. funciona.
while True:
    agent()

# ...aí você vai colocando coisa:
while not done:
    goal    = parse("/goal")
    thought = llm(state, goal)
    action  = parse_action(thought)
    obs     = tools.run(action)
    state   = compact(state, obs)
    if budget_blownd: break
```

02

PARTE 02 / 05

agente vs humano.

o debate que
divide a
comunidade.
spoiler: os dois
vencem.

// PADRÃO POR TRÁS DOS QUATRO CASOS

não é o **m**odelo. **n**ão é o **p**rompt.

amplitude ×
profundidade

amplitude = o que a máquina cobre
enquanto você dorme.

profundidade = a hipótese ousada
que só você teve.

o resto é detalhe de implementação.

// DUAS FILOSOFIAS

solta o bicho vs dirige você mesmo.

agente autônomo

- solta de noite, volta de manhã.
- cobre área impossível pra humano.
- custa *tokens*, não tempo.
- produz *ruído*. muito.
- ótimo pra programas grandes, codebases novas, fuzzing assistido.

human-in-the-loop

- você dirige. modelo é copiloto.
- cada passo passa por *julgamento*.
- custa seu tempo. menos tokens.
- produz *signal*. consistente.
- ótimo pra hipótese ousada, exploit complexo, decisões irreversíveis.

// BROAD-FIRST — FORÇA DO AGENTE

cobrir tudo, ranquear depois.

ENDPOINTS VARRIDOS

200+

em paralelo

LINHAS AUDITADAS

~50k

com triagem ranqueada

BINÁRIOS REVISADOS

10

em uma noite

o agente não pensa melhor que você.

ele só paraleliza o tédio.

custo real: tokens + curadoria humana do output.

// DEPTH-FIRST – ONDE VOCÊ AINDA MANDA

a intuição não terceiriza.

aquele caminho
que cheira mal
e ninguém mais sentiu.

→ hipótese ousada que **contraria** o consenso da codebase.

→ "e se eu jogar isso aqui contra aquilo lá?" – humano.

→ modelo não tem coragem. ele otimiza pra média.

a vantagem é sua. ainda.

// MODELO MENTAL

o agente é seu estagiário.

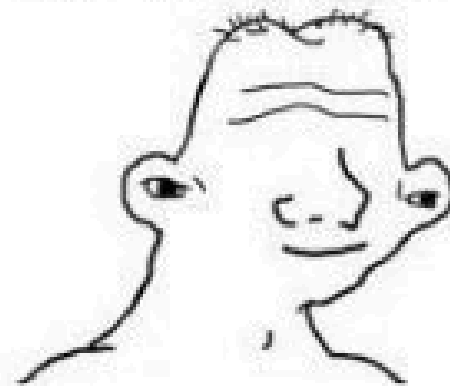
você passa o trabalho chato – o
que você não tem saco de fazer na
mão.

- testar a porra toda que você não tem saco de testar
- validar uma teoria – rápido, sem cerimônia
- provar que algo não dá pra bypassar
- codar harness de fuzzing
- codar os testezinhos chatos

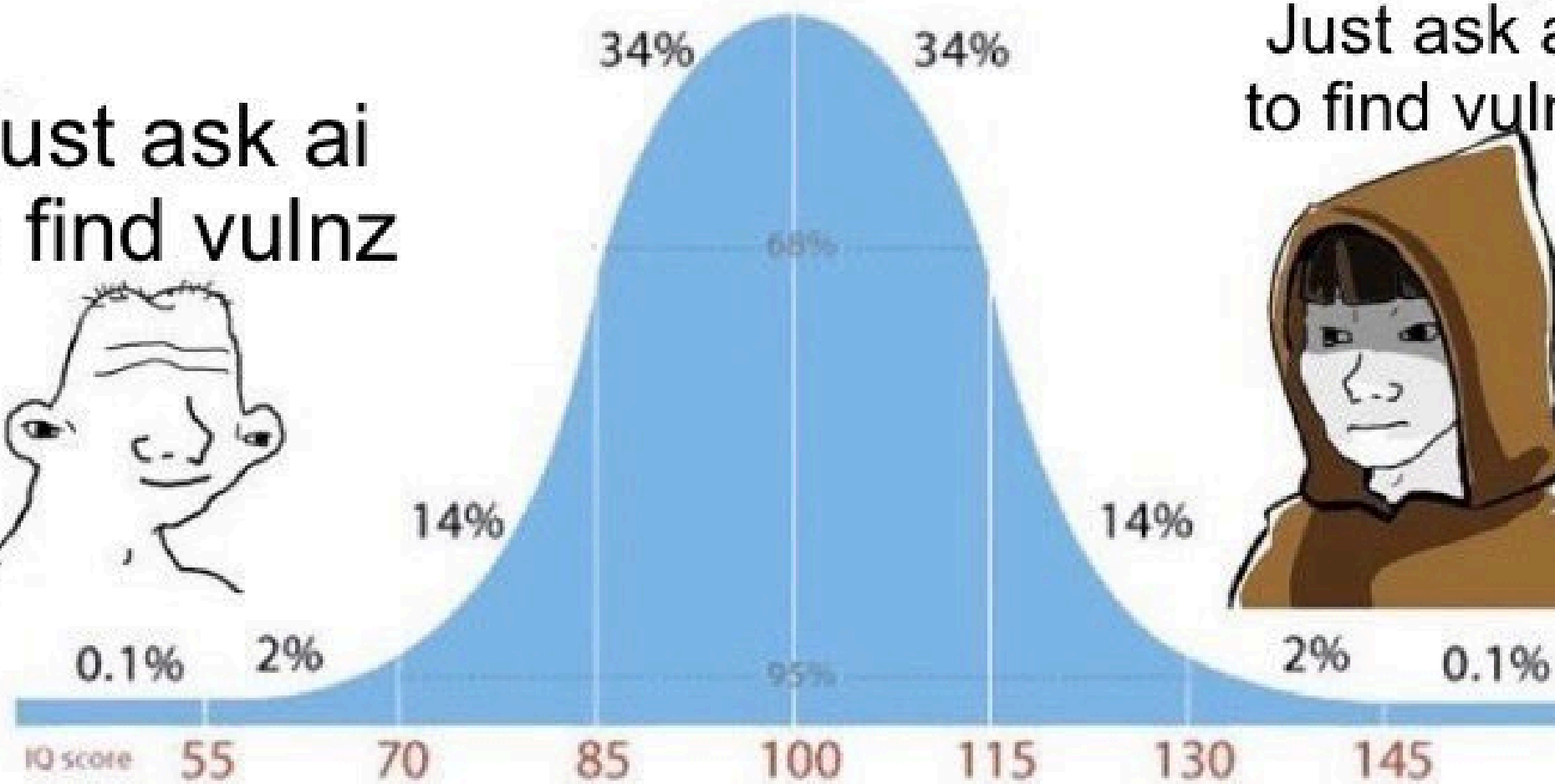
Nooo you need to run
multiple agents and connect with
xyz mcps and run in a pipeline



Just ask ai
to find vulnz



Just ask ai
to find vulnz



// TUDO MUDA TODA SEMANA

Não cace o próprio rabo

o resto

caçando o próprio
rabo

harness novo

framework +0.5%

modelo +2%

reescreve tudo

de novo...

hack?

eu

ferramenta
sedimentada

tool estável

HACKEAR

o framework novo funciona 0,5% melhor. o
modelo novo, 2%. **e muda tudo na semana que
vem.**

não adianta caçar o próprio rabo – pega a
ferramenta que já funciona e vai hackear.

// COMO VOCÊ OLHA O ALVO

static vs dynamic.

static

- o modelo lê código **muito bem**
- rápido • **zero ferramental**
- meu default hoje – uns 80%

dynamic

- ainda precisa de muito ferramental
- não é rápido
- mas funciona: **extensão de chrome • emulador android**

// É SEMPRE O MESMO TRADE-OFF

o mesmo eixo, vários nomes.

generalista ↔ **especialista**

CAÇA

broad-first ↔ **depth-first**

BUG BOUNTY

automação ↔ **manual**

ANÁLISE

static ↔ **dynamic**

cobre tudo & raso • ou foca & fundo. a escolha é tempo × conhecimento.

03

PARTE 03 / 05

anatomia de uma análise.

quatro fases,
lendo código.
cada uma muda
com o modelo no
loop.

// FASE 1

mapear — a superfície.

01

ANTES

clona o repo, lê README, abre 40 arquivos pra achar os entrypoints na mão.

monta o mapa de cabeça. cada vez do zero.

COM VIBE

agente lê a base e devolve o mapa: entrypoints, fluxo de dados, deps podres, superfície ranqueada.

"auth nessas 3 arquivos. parsing de input nessas 5."

ganho: horas de leitura cega.

// FASE 2 – ONDE MAIS GENTE ERRA

triagem – onde olhar.

02

PERGUNTA ERRADA

"lê esse código e me diz se tem bug."

(50 'maybe' inúteis. puro ruído.)

PERGUNTA CERTA

"me mostra as 30 funções que tocam input do usuário sem sanitizar – e por quê."

a pergunta é a skill.

// FASE 3 – HITL OBRIGATÓRIO

hipótese — source → sink.

03

AGENTE SOZINHO

inventa o data flow.

inventa a invariante.

inventa o CVE.

resultado: lixo confiante.

COM HUMANO DIRIGINDO

modelo **traça o caminho**: de onde vem o input, por onde passa, onde explode.

você decide se o path é alcançável e vale o exploit.

// FASE 4

prova – chega no sink?

04

ANTES

horas escrevendo harness só pra alcançar a função.

lutando com build, stubs, dependência.

30min formatando o input.

COM VIBE

modelo escreve o PoC e os stubs, prova que o input chega no sink.

encoding, framing, setup – **tédio mecânico.**

você revisa, confirma, reporta.

04

PARTE 04 / 05

exemplos do mundo real.

0day · bug
bounty · CTF ·
pentest – quatro
contextos, um
padrão.

// EXEMPLO 3 / 4

CTF

CTF onde as LLMs gabaritavam tudo **one-shot**. Veio um web com ENV var injection numa lib grande e específica – aí o jogo virou força bruta inteligente.

DESAFIO web • ENV var injection numa lib grande

HIPÓTESE alguma das 100+ ENVs lê arquivo arbitrário

SETUP IA parseia, lê e categoriza todas as ENVs do código

RESULTADO **testou uma a uma até uma ler a /flag**

MORAL não foi esperto – foi exaustivo onde você não teria saco

o HMAC que o .replace() entregou.

```
• • • IMAGING.PY
```

```
url = url.replace(f"/{hash}/", "")  
      .replace(f"/{quoted}/", "") # tira TUDO
```

```
# URL maliciosa — hash injetada 2x:  
/HASH/300x200/s3.glbimg.co/HASH/m/...
```

```
↳ assinatura válida, carrega de s3.glbimg.co
```

Thumbor assina cada URL com **HMAC**. pra validar, ele tira a hash da URL com `.replace()` — duas vezes (crua e url-encoded).

o bug: `.replace()` tira TODAS as ocorrências, não a primeira. injeta a hash de novo no meio da URL → continua válida, mas o **domínio carregado muda**.

a IA exauriu outros caminhos e achou esse. pedi pra explicar call-by-call e guiei ela a fuzzar a função até o ângulo do domínio.

// 0DAY • THUMBOR — LOW LEVEL

int32 overflow → type confusion.

os filtros em C usam int32 pras dimensões. estoura o int → offset negativo → escreve **antes** do buffer na heap.

o buffer é um **PyObject**. sobrescreve o ponteiro ob_type → type confusion → RCE (teórico).

não saiu one-shot — ASLR, blend math (~0.8% dos layouts), sem infoleak. mas mapeamos os caminhos viáveis.

// a IA achou e categorizou TODO sink com o mesmo padrão

`_composite.c`

`_convolution.c`

`_alpha.c`

`_fill.c`

`_curve.c`

`_bounding_box.c`

`wavelet_sharpen.c`

• • • _COMPOSITE.C:45

```
int off = (y * width * num_bytes);  
# int32 × int32 → estoura → off negativo  
aux1[off] = pixel; # write antes do buffer
```

// 0DAY • THUMBOR — O QUE A IA CATALOGOU

um repo, vários bugs.

FILTRO	BUG	POR QUE PAROU
<code>_alpha.c</code>	loop infinito (mode vazio)	mode vem do PIL
<code>_fill.c</code>	divisão por zero	buffer do HTTP nunca vazio
<code>_convolution.c</code>	00B (buffer mismatch)	PIL normaliza as dimensões
<code>_bounding_box.c</code>	integer overflow	não alcançável via HTTP
<code>_curve.c</code>	truncation uchar	heap corruption "cega"
<code>wavelet_sharpen.c</code>	heap 00B (hat_transform)	leak vira float → satura
<code>_composite.c</code>	int overflow → write 00B	✓ o explorável

// EXEMPLO 2 / 4

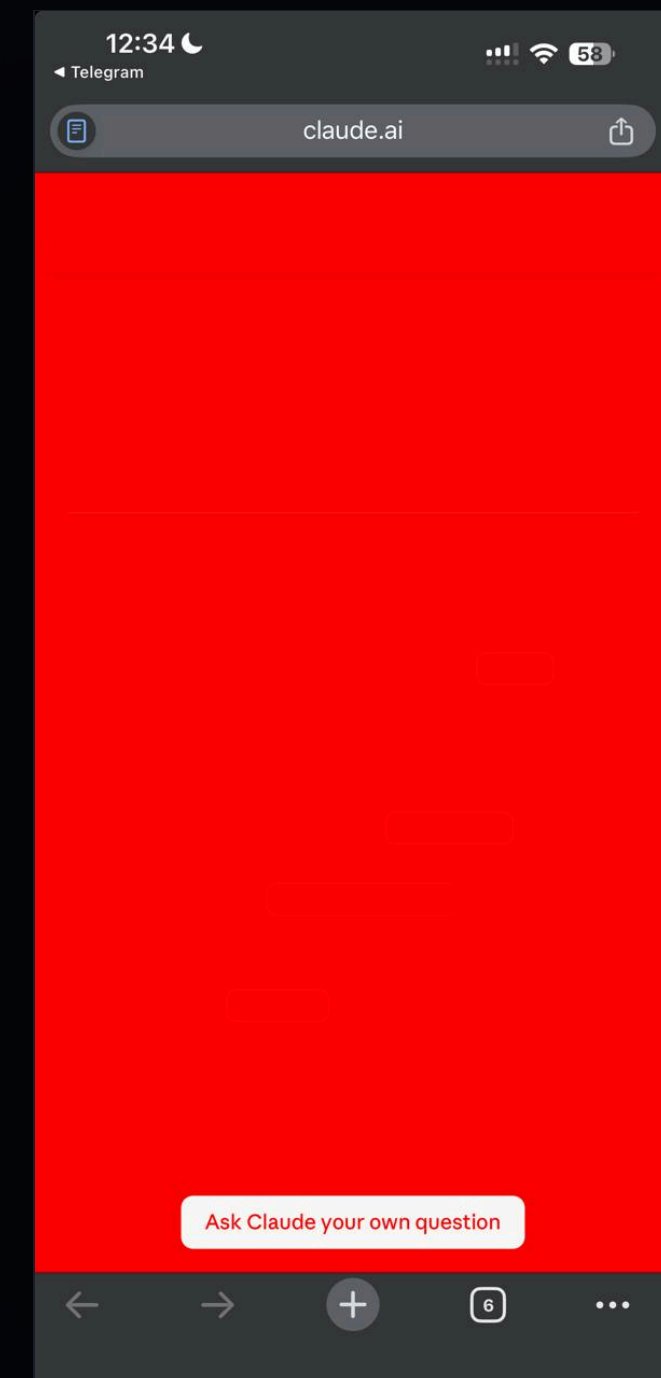
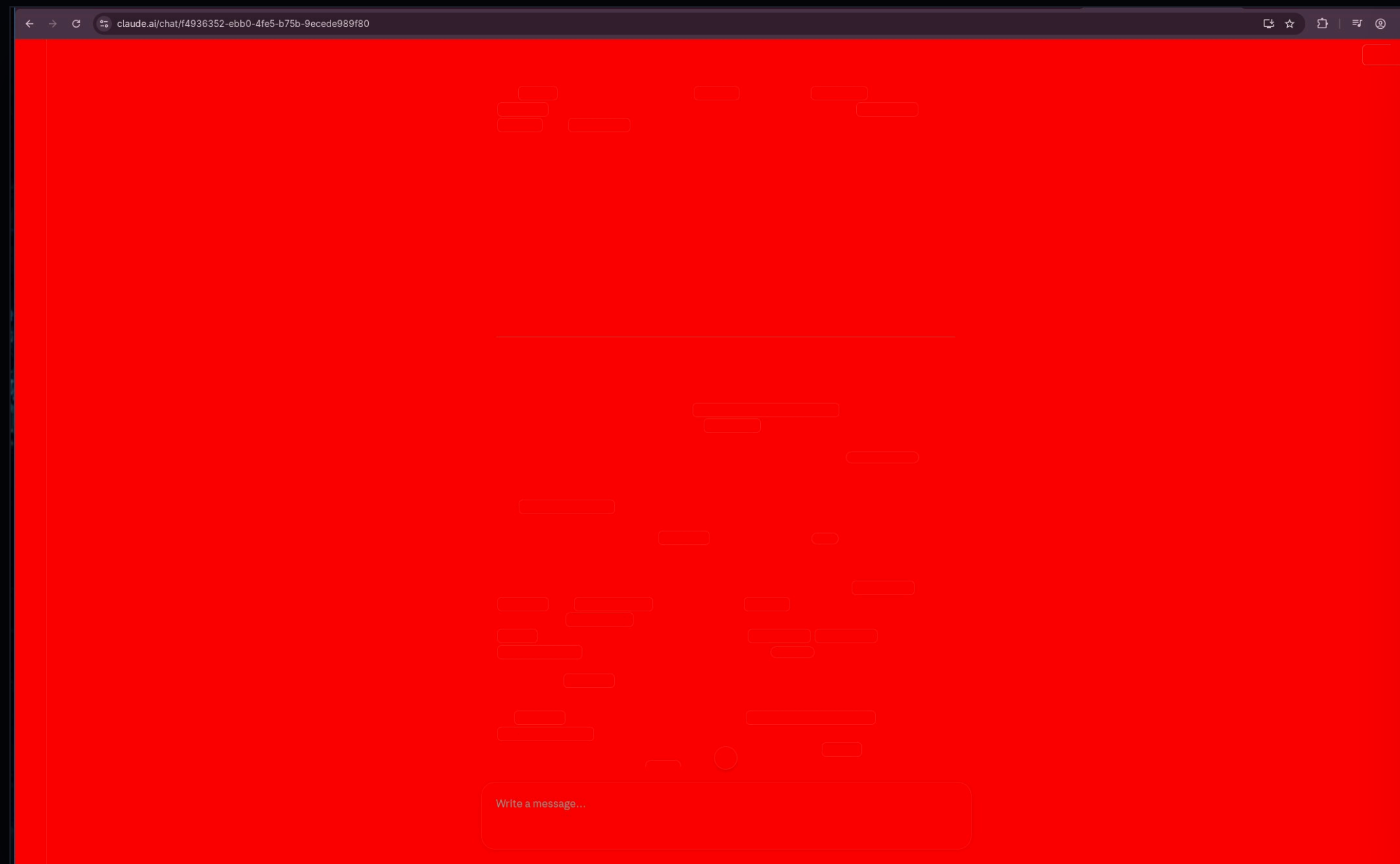
bug bounty

Site usando **KaTeX** + **Mermaid**. IA juntou os payloads de todas as CVEs recentes, montou um testbed pra fuzzar e criar variantes dos patches. No caminho, o **Claude se hackeou**.

ALVO	app renderizando KaTeX + Mermaid de input do usuário
HIPÓTESE	as CVEs recém-corrigidas têm variante – o patch é estreito
SETUP	IA junta os payloads, monta testbed, fuzza e analisa os patches
RESULTADO	19 findings novos no mermaid 11.15.0 (último patch) · 3+ 0days
MORAL	CSS injection escapou o scoping → pintou o claude.ai de vermelho

// BUG BOUNTY • 0 CLAUDE SE HACKEOU

CSS injection > scoping do Mermaid.



05

PARTE 05 / 05

demo ao vivo.

terminal,
modelo, target
escolhido por
você. sem rede.

// SE VOCÊ SÓ LEMBRAR DE TRÊS COISAS

Leva isso daqui.

01 a fórmula é **amplitude × profundidade**. resto é detalhe.

02 invista em **harness e skills**, não em prompt mágico.

03 em 2027 quem ignorar isso vai estar caçando **bugs que o agente já reportou**.



PERGUNTAS, PROVOCAÇÕES, OFENSAS.

Caio Lüders · @caioluders

harness + skills no github: [github.com/\[user\]/vibehack](https://github.com/caioluders/vibehack)

obrigado, dunasec.